

GraphOne / FrontierAtlas Intelligence Graph

Phase VI: Technical Architecture & Production Design Document

1. Executive Architecture Overview

GraphOne / FrontierAtlas powers the global Intelligence Graph for the artificial intelligence and venture capital ecosystem. The platform ingests, resolves, normalizes, and enriches entities across startups, products, research papers with GitHub metrics, 24-hour job signals, and real-time news feeds. The architecture is designed for fault tolerance, anti-bot resilience, zero hallucination, and horizontal scalability.

2. Scale Strategy: Acquiring 500,000+ Records Automatically

Scaling from thousands to 500,000+ records without manual intervention or architectural refactoring is achieved via a decoupled, event-driven distributed system:

- ? **Distributed Partitioned Queues (Kafka / RabbitMQ)**: Crawl tasks are partitioned by domain hash (hash(domain) % partitions). This prevents thundering herds on individual web hosts while maximizing parallel throughput across independent domains.
- ? **Stateless Async Workers (Ray / Celery on Kubernetes)**: Independent crawler nodes run asyncio + aiohttp / Playwright Async. Cluster size automatically expands via Kubernetes HPA based on queue backpressure.
- ? **Distributed Bloom Filters (Redis O(1) Lookups)**: Scalable Redis Bloom filters verify in sub-millisecond time whether a URL or entity has been processed, eliminating redundant network hops and socket exhaustion.
- ? **Micro-Batch Bulk Ingestion**: Extracted records stream into persistent storage in micro-batches (500 records per transaction), eliminating database memory spikes.

Component	1,000 Records (Trial)	500,000+ Records (Production Scale)
Worker Pool	Async Local Pool (25 workers)	Ray Distributed Cluster (128 Pods)
Task Distribution	Asyncio In-Memory Queue	Apache Kafka (32 Partitions)
Deduplication	In-Memory Set + SQLite Unique	Redis Scalable Bloom Filters + Sharded PostgreSQL
Anti-Bot / Proxy	Stealth UA + Header Rotation	Residential Proxy Pool + Camoufox Headless Fleet

3. Managing Context Windows (413s) & Rate Limits (429s)

LLM-based entity extraction at scale faces two critical operational hurdles: token payload overflows (413) and rate limits (429).

A. 413 Payload Too Large Elimination Strategy:

? **Boilerplate Stripping:** The ContentChunker removes DOM clutter, scripts, styles, and navigation tags, condensing raw web pages by 75-85% while preserving dense semantic data.

? **Head-and-Tail Truncation (70/30 Rule):** For lengthy technical papers and articles, the engine retains the top 70% (abstract, introduction, metadata) and bottom 30% (conclusions, pricing, contact), capping context to 10,000 characters.

? **Deterministic NLP Fallback:** If raw payloads exceed model limits, Tier 4 deterministic heuristic extraction parses structured data with zero model latency.

B. 429 Rate Limit Handling & Fallback Routing Chain:

? **Multi-Tier Fallback Chain:** Automatic cascading failover: Gemini 1.5 Flash -> Groq Llama 3 -> DeepSeek Chat -> Deterministic Rule Parser.

? **Full Jitter Exponential Backoff:** Retries use randomized jitter to prevent synchronous re-try floods.

? **Leaky Bucket Rate Limiters:** Client-side token buckets throttle requests before hitting provider rate limits.

4. Freshness Tracking: Guarantees for 24-Hour Real-Time Signals

To ensure all ingested job and news signals are strictly published within the last 24 hours without duplicate processing:

? **Strict UTC ISO-8601 Normalization:** Relative dates ('2 hours ago', 'yesterday') and missing meta tags are normalized to UTC ISO-8601 timestamps and filtered with $0 \leq (\text{now} - \text{pub_date}) \leq 24\text{h}$.

? **Distributed Redlock Synchronization:** Atomic Redis locks prevent multiple crawler nodes from fetching the same fresh article or job URL simultaneously.

? **HTTP ETag / Last-Modified Caching:** Polling feeds use conditional HTTP headers, skipping re-downloads when servers return 304 Not Modified.

? **Content Fingerprinting:** SHA-256 checksums on canonical title + company ensure identical postings with differing tracking URLs are deduped.

5. Storage Strategy: Polyglot Persistence for Knowledge Graphs

No single database fulfills high-write transactional ingestion, deep multi-hop graph traversals, and semantic vector similarity search. The architecture deploys a polyglot storage layer:

Storage Layer	Technology	Operational Responsibility & Rationale
Primary ACID Store	PostgreSQL 16	Stores raw & normalized entity tables, enforces JSON Schema validations, manages audit logs, provides transactional consistency.
Knowledge Graph	Neo4j Enterprise	Models venture & AI graph: (Startup)-[:MAKES]->(Product), (Author)-[:AUTHORED]->(Paper)-[:HAS_REPO]->(GitHub). Enables sub-second multi-hop graph queries.
Vector Search	Qdrant / pgvector	Indexes embeddings of paper abstracts, startup descriptions, and product value propositions for semantic discovery and deduplication.
Cache & Locks	Redis Cluster	Distributed locks (Redlock), token bucket rate limiting, and global Bloom filter deduplication.

6. Deterministic Entity Resolution Pipeline

The entity resolution engine normalizes noisy scraped organization names (e.g., 'OpenAI Inc', 'Open AI', 'OpenAI OpCo LLC' -> 'OpenAI') through a 4-tier deterministic pipeline:

- Exact Alias Indexing:** Sub-millisecond dictionary lookup against 50+ canonical seed organizations and known aliases.
- Legal Suffix Stripping:** Regex-based stripping of designations (Inc, LLC, Ltd, Corp, Technologies, Labs, PBC, GmbH, SAS).
- Fuzzy Similarity Matching:** RapidFuzz Jaro-Winkler and Token Sort Ratio matching with a strict confidence threshold (score >= 0.88).
- Audit Logging:** Every single normalization transformation is recorded in the Entity Mapping Log with raw name, canonical name, confidence score, and resolution method.